

OBJECT ORIENTED PROGRAMMING [CT-260]



NAME: ABDUL RAHEEM SHEIKH

ROLL NO: CT-25192

DEPARTMENT: BCIT

BATCH: 2025

YEAR & SECTION: FSCS-D

LAB: 3

CODES

```
1 //1. Write a C++ program to copy the value of one object to another object using copy
2 //constructor. For example you can define a class for complex number and create its
3 //object for performing this task. Remember that you must allocate memory
4 //dynamically to data members.
5 #include <iostream>
6 using namespace std;
7
8 class Complex {
9 private:
10     int *even;
11     int *odd;
12
13 public:
14     // Parameterized Constructor
15     Complex(int e, int o) {
16         even = new int;
17         odd = new int;
18         *even = e;
19         *odd = o;
20     }
21
22     // Copy Constructor
23     Complex(const Complex &c) {
24         even = new int;
25         odd = new int;
26         *even = *(c.even);
27         *odd = *(c.odd);
28     }
29
30     void display() {
31         cout << "Complex Number: " << *even << " + " << *odd << "i" << endl;
32     }
33     // Destructor
34     ~Complex() {
35         delete even;
36         delete odd;
37     }
38 };
39
40 int main() {
41     // Constructor called
42     Complex c1(2, 5);
43     // Copy constructor called
44     Complex c2(c1);
45
46     c1.display();
47     c2.display();
48     return 0;
49 }
```

OUTPUT

Complex Number: $2 + 5i$
Complex Number: $2 + 5i$

```
1 //2. In a virtual battle arena game called "Epic Clash," players control powerful
2 //characters to engage in intense battles against each other. Each character has
3 //distinct abilities and attributes, including health, attack power, and defense. Your
4 //task is to implement the Character class to encapsulate these attributes, provide
5 //getter and setter methods for them, and offer different constructors to create
6 //characters with various starting conditions.
7 //Encapsulation: Ensure that the character's attributes (health, attackPower, and
8 //defense) are private to the Character class, accessible only through appropriate
9 //getter and setter methods, allocated memory dynamically in heap.
10 //Constructors: Implement three constructors: A default constructor that initializes a
11 //character with standard starting values for health, attack power, and defense. A
12 //parameterized constructor that allows specifying custom values for health, attack
13 //power, and defense. A copy constructor that creates a new Character object by
14 //copying the data from an existing Character object.
15 #include <iostream>
16 using namespace std;
17
18 class Character {
19 private:
20     int *health;
21     int *attackPower;
22     int *defense;
23
24 public:
25     // Default Constructor
26     Character() {
27         health = new int(90);
28         attackPower = new int(40);
29         defense = new int(20);
30     }
31
32     // Parameterized Constructor
```

```

32 // Parameterized Constructor
33 Character(int h, int a, int d) {
34     health = new int(h);
35     attackPower = new int(a);
36     defense = new int(d);
37 }
38 // Copy Constructor
39 Character(const Character &c) {
40     health = new int(*(c.health));
41     attackPower = new int(*(c.attackPower));
42     defense = new int(*(c.defense));
43 }
44
45 // Getters
46 int getHealth() { return *health; }
47 int getAttack() { return *attackPower; }
48 int getDefense() { return *defense; }
49 // Setters
50 void setHealth(int h) { *health = h; }
51 void setAttack(int a) { *attackPower = a; }
52 void setDefense(int d) { *defense = d; }
53 void display() {
54     cout << "Health: " << *health
55          << ", Attack: " << *attackPower
56          << ", Defense: " << *defense << endl;
57 }
58 // Destructor
59 ~Character() {
60     delete health;
61     delete attackPower;
62     delete defense;
63 }
64 };
65 int main() {
66     Character c1;
67     Character c2(130, 100, 140);
68     Character c3 = c2;
69     c1.display();
70     c2.display();
71     c3.display();
72     return 0;
73 }

```

OUTPUT

```

Health: 90, Attack: 40, Defense: 20
Health: 130, Attack: 100, Defense: 140
Health: 130, Attack: 100, Defense: 140

```

```

1 //3. Create a class tollbooth. The two data items are a type int to hold the total number
2 //of cars and a type double to hold the total amount of money collected. A
3 //constructor initializes both these to 0. When a car passes the toll, a member
4 //function called payingCar( ) increments the car total and adds 0.50 to the cash
5 //total. Another member function displays the two totals. DESIGN and IMPLEMENT
6 //this case. Make assumptions (if required) and include it in the description before
7 //designing the solution.
8 #include <iostream>
9 using namespace std;
10
11 class TollBooth {
12 private:
13     int totalCars;
14     double totalCash;
15
16 public:
17     // Default Constructor
18     TollBooth() {
19         totalCars = 0;
20         totalCash = 0.0;
21     }
22
23     void payingCar() {
24         totalCars++;
25         totalCash += 0.50;
26     }
27
28     void display() {
29         cout << "Total Cars: " << totalCars << endl;
30         cout << "Total Cash: Rs. " << totalCash << endl;
31     }
32 };
33
34 int main() {
35     TollBooth booth;
36
37     booth.payingCar();
38     booth.payingCar();
39     booth.payingCar();
40
41     booth.display();
42     return 0;
43 }

```

OUTPUT

```

Total Cars: 3
Total Cash: Rs. 1.5

```

```

1 //4. Some of the characteristics of a book are the title, author(s), publisher, ISBN, price,
2 //and year of publication. Design a class bookType that defines the book as an ADT.
3 //? Each object of the class bookType can hold the following information about a
4 //book: title, up to four authors, publisher, ISBN, price, and number of copies in
5 //stock. To keep track of the number of authors, add another member variable.
6 //? Include the member functions to perform the various operations on objects of
7 //type bookType.
8 //For example, the usual operations that can be performed on the title are to
9 //show the title, set the title, and check whether a title is the same as the actual
10 //title of the book. Similarly, the typical operations that can be performed on the
11 //number of copies in stock are to show the number of copies in stock, set the
12 //number of copies in stock, update the number of copies in stock, and return the
13 //number of copies in stock. Add similar operations for the publisher, ISBN, book
14 //price, and authors. Add the appropriate constructors and a destructor (if one is
15 //needed).
16 //? Write the definitions of the member functions of the class bookType.
17 //? Write a program that uses the class bookType and tests various operations on
18 //the objects of the class bookType. Declare an array of 100 components of type
19 //bookType. Some of the operations that you should perform are to search for a
20 //book by its title, search by ISBN, and update the number of copies of a book.
21 #include <iostream>
22 #include <string>
23 using namespace std;
24
25 class bookType {
26 private:
27     string title;
28     string authors[4];
29     int authorCount;
30     string publisher;
31     string ISBN;
32     double price;
33     int copies;
34
35 public:
36     // Default Constructor
37     bookType() {
38         authorCount = 0;
39         price = 0.0;
40         copies = 0;
41     }
42
43     // Set functions
44     void setTitle(string t) {
45         title = t;
46     }
47
48     void addAuthor(string a) {
49         if (authorCount < 4)
50             authors[authorCount++] = a;
51     }
52
53     void setPublisher(string p) {
54         publisher = p;
55     }
56
57     // Show functions
58     void showTitle() const {
59         cout << "Title: " << title << endl;
60     }
61
62     void showAuthors() const {
63         cout << "Authors: ";
64         for (int i = 0; i < authorCount; i++)
65             cout << authors[i] << " ";
66         cout << endl;
67     }
68
69     void showPublisher() const {
70         cout << "Publisher: " << publisher << endl;
71     }
72
73     void showISBN() const {
74         cout << "ISBN: " << ISBN << endl;
75     }
76
77     void showPrice() const {
78         cout << "Price: " << price << endl;
79     }
80
81     void showCopies() const {
82         cout << "Copies: " << copies << endl;
83     }
84
85     // Update functions
86     void updateCopies(int n) {
87         copies = n;
88     }
89
90     // Search functions
91     bool searchTitle(string t) const {
92         return title == t;
93     }
94
95     bool searchISBN(string isbn) const {
96         return ISBN == isbn;
97     }
98
99     // Destructor
100    ~bookType() {
101        // No action needed here
102    }
103 };

```

```

49         if (authorCount < 4)
50             authors[authorCount++] = a;
51     }
52
53     void setPublisher(string p) {
54         publisher = p;
55     }
56
57     void setISBN(string i) {
58         ISBN = i;
59     }
60
61     void setPrice(double pr) {
62         price = pr;
63     }
64
65     void setCopies(int c) {
66         copies = c;
67     }
68
69     // Get functions
70     string getTitle() {
71         return title;
72     }
73
74     string getISBN() {
75         return ISBN;
76     }
77
78     int getCopies() {
79         return copies;
80     }
81
82     // Update copies
83     void updateCopies(int c) {
84         copies += c;
85     }
86
87     // Display book information
88     void display() {
89         cout << "Title: " << title << endl;
90         cout << "Authors: ";
91         for (int i = 0; i < authorCount; i++)
92             cout << authors[i] << " ";
93         cout << endl;
94         cout << "Publisher: " << publisher << endl;
95         cout << "ISBN: " << ISBN << endl;
96         cout << "Price: " << price << endl;
97         cout << "Copies in stock: " << copies << endl;
98         cout << "-----" << endl;
99     }
100 };

```

```

102 int main() {
103     bookType books[100];
104
105     // Adding one book
106     books[0].setTitle("C++ Programming");
107     books[0].addAuthor("Bjarne Stroustrup");
108     books[0].setPublisher("Pearson");
109     books[0].setISBN("ISBN123");
110     books[0].setPrice(600);
111     books[0].setCopies(10);
112
113     // Search book by title
114     string searchTitle = "C++ Programming";
115     for (int i = 0; i < 1; i++) {
116         if (books[i].getTitle() == searchTitle)
117             books[i].display();
118     }
119
120     // Search book by ISBN
121     string searchISBN = "ISBN123";
122     for (int i = 0; i < 1; i++) {
123         if (books[i].getISBN() == searchISBN) {
124             books[i].display();
125         }
126     }
127
128     // Update number of copies
129     books[0].updateCopies(5);
130
131     cout << "After updating copies:" << endl;
132     books[0].display();
133
134     return 0;
135 }
136
137

```

OUTPUT

Title: C++ Programming
Authors: Bjarne Stroustrup
Publisher: Pearson
ISBN: ISBN123
Price: 600
Copies in stock: 10

Title: C++ Programming
Authors: Bjarne Stroustrup
Publisher: Pearson
ISBN: ISBN123
Price: 600
Copies in stock: 10

After updating copies:
Title: C++ Programming
Authors: Bjarne Stroustrup
Publisher: Pearson
ISBN: ISBN123
Price: 600
Copies in stock: 15